

Solving the capacitated vehicle routing problem with polygonal neighborhoods using a metaheuristic enhanced with block coordinate descent*

Ernesto G. Birgin[†] Rogério S. Matos[‡]

June 29, 2026

Abstract

We study a capacitated vehicle routing problem (CVRP) in which customers are represented by planar regions, called neighborhoods, rather than fixed points. Visiting a customer consists of selecting a point within its neighborhood. Therefore, besides determining the sequence of customers visited by each vehicle, one must also determine the corresponding visit points. To better reflect practical applications, neighborhoods are allowed to be arbitrary polygons, either convex or non-convex, and distances are measured using the Euclidean norm. Applications include aerial reconnaissance, wireless sensor data collection and inspection, and two-stage delivery systems. We propose a general variable neighborhood search framework in which each neighbor is represented by a collection of routes. Constructing a route involves two stages: defining a customer sequence and then determining the associated visit points. The latter task is solved by a block coordinate descent (BCD) procedure, which efficiently computes high-quality routes. The computational efficiency of BCD enables extensive experiments on both newly generated and adapted benchmark instances. The small- and medium-scale testbed contains 134 instances, of which 73 are solved to optimality by CPLEX. The proposed framework matches all proven-optimal solutions and matches or improves the best solutions obtained by CPLEX for all remaining instances. For 106 large-scale instances adapted from the CVRP literature, CPLEX proves optimality for only three instances. The proposed framework matches these solutions and matches or improves the best solutions obtained by CPLEX for all remaining instances. Furthermore, it improves a CVRP-based upper bounds in every instance.

Keywords: vehicle routing problem, capacity constraints, polygonal neighborhoods, block coordinate descent, general variable neighborhood search.

*This work has been partially supported by the Brazilian agencies FAPESP (grants 2013/07375-0, 2022/05803-3, 2023/08706-1, and 2024/22384-0) and CNPq (grant 302073/2022-1).

[†]Department of Computer Science, Institute of Mathematics and Statistics, University of São Paulo, Rua do Matão, 1010, Cidade Universitária, 05508-090, São Paulo, SP, Brazil. e-mail: egbirgin@ime.usp.br. Corresponding author.

[‡]Department of Applied Mathematics, Institute of Mathematics and Statistics, University of São Paulo, Rua do Matão, 1010, Cidade Universitária, 05508-090, São Paulo, SP, Brazil. e-mail: rogerio.matos@ime.usp.br

1 Introduction

In the classic vehicle routing problem (VRP) [13] (see also [20, 27, 41]), there are m customers, a depot, and an unlimited number of identical trucks. The problem consists in building a set of routes such that each route starts and ends at the depot and, as a whole, visit each customer exactly once. The combined length of the set of routes must be minimized. Note that in the classical problem customers have fixed geographic locations such that the distance between each pair of customers or between a customer and the depot is known in advance and fixed. Thus, the length of a route that leaves the depot, visits customers $i_1, i_2, \dots, i_\kappa$, and returns to the depot is given by the simple sum of the distance from the depot to customer i_1 , the distances between consecutive customers along the route, and the distance between customer i_κ and the depot.

In the VRP with neighborhoods (VRPN), each customer is represented by an arbitrary region of the plane, termed neighborhood, rather than by a coordinate location point. This idea was introduced in the context of the TSP in [4]. In this context, visiting a customer means visiting a point within the customer’s neighborhood. In addition to the challenges of the VRP, the visit point for each customer must be determined. Furthermore, since the location of visit points is a variable in the problem, the distances between pairs of customers and between the customers and the depot are not fixed. In the specific case when neighborhoods are given by two-dimensional balls (circles), the problems are known as Close Enough TSP (CETSP) [5, 47, 9, 10, 12, 15, 16, 19, 21, 42, 46, 48] or Close Enough VRP (CEVRP) [32, 35]. The names come from the fact that the radius of a customer’s circle is interpreted as the maximum Euclidean distance that the route can pass for the customer to be considered as visited. To the authors’ knowledge, only versions with neighborhoods given by balls have been considered for the VRP [32, 35]; in these studies, applications in aerial reconnaissance route planning are highlighted.

In the present work, we consider a VRPN in which each customer’s neighborhood is represented by an arbitrary non-convex simple polygon. See Figure 1 for an illustrative example. We consider that each customer i has a demand $d_i > 0$ and that all trucks have the same maximum capacity $\mathcal{C}$. Therefore, we call it the capacitated VRPN (CVRPN) problem. We also consider the Euclidean norm as a measure of distance; this implies that the problem is symmetric. The CVRPN that we study here includes the practical case in which trucks first deliver the aggregate demand of a neighborhood to a drop-off point located anywhere within that neighborhood. Crowdsourced couriers then complete the last mile [2, 29]. Each courier operates within a specific region represented by a non-convex polygon reflecting realistic urban boundaries or accessibility constraints. The problem under consideration also encompasses scenarios in which vehicles are required to visit customers, not to deliver goods, but rather to perform measurements, such as collecting or inspecting data via wireless sensors. This technology, called automated meter reading (AMR) [17, 37], is used by utility service providers, such as electricity, water, and gas. It enables remote monitoring, billing, and management. In this scenario, it is unnecessary for the vehicle to reach the customer’s exact location; it suffices for the vehicle to pass close enough for the measurement to be completed. Although such proximity is often modeled as a circular region around the customer, signal propagation may be obstructed in practice by walls, buildings, or other obstacles [24], resulting in irregular coverage zones. Consequently, the effective measurement region is more accurately represented by a non-convex polygon, which captures

the true spatial constraints imposed by the environment on the quality of the signal transmission. Finally, the CVRPN with (convex) polygonal neighborhoods corresponds to the CEVRP in which the distances that define the neighborhoods are measured using the Manhattan or Chebyshev norms.

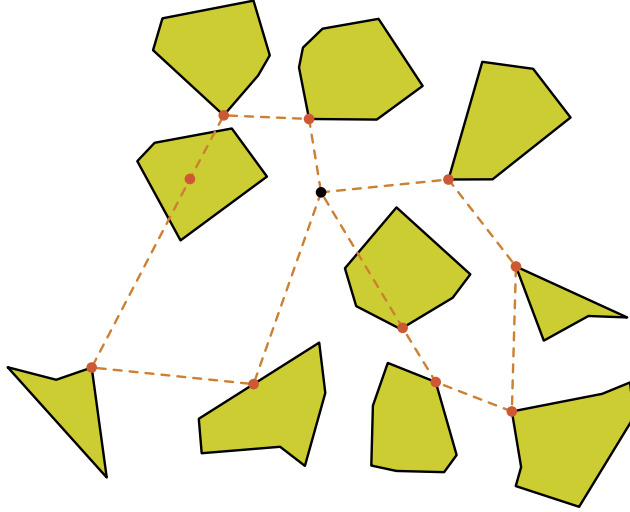


Figure 1: Example of a two-route optimal solution to a ten-customer instance of the capacitated vehicle routing problem with non-convex polygonal neighborhoods. Note that each customer’s visit point may be at a vertex, on an edge, or even within the interior of the polygon representing the customer’s neighborhood.

We say that a sequence that starts at the depot, is followed by a sequence of customers $i_1, i_2, \dots, i_\kappa$ and ends at the depot, is a *visit sequence*. In order for a visit sequence to become a route, we need to find the visit points for each of the customers $i_1, i_2, \dots, i_\kappa$. In order for the route to have minimal length, the visit points must be a solution to the problem

$$\underset{\{p_{i_j} \in \mathbb{R}^2, j=1, \dots, \kappa\}}{\text{Minimize}} \sum_{j=0}^{\kappa} \|p_{i_{j+1}} - p_{i_j}\| \text{ subject to } p_{i_j} \in P_{i_j} \text{ for } j = 1, \dots, \kappa. \quad (1)$$

In (1), p_{i_0} and $p_{i_{\kappa+1}}$ both represent the given geographical position of the depot, P_{i_j} is the polygon representing the neighborhood of customer i_j , and the variable p_{i_j} is the visit point, to be determined, of customer i_j . If the polygons representing the neighborhoods were all convex, problem (1) would be a nonlinear optimization problem. When non-convex polygons exist, problem (1) can be reformulated as a problem with a linear objective function, a convex feasible region determined by linear and quadratic constraints, and continuous and binary variables. This kind of problem is known as mixed-integer quadratically constrained programming (MIQCP). Commercial exact solvers such as CPLEX and Gurobi offer features to search for the global solution to this class of problems. In this paper, we propose solving problem (1) with a block coordinate descent method [3, 7, 45].

The flexibility gained by replacing fixed customer locations with polygonal neighborhoods comes at a significant computational cost. For a given set of routes, the corresponding visit points must also be determined, and these visit-point decisions directly affect the route lengths. Consequently, evaluating a candidate solution requires determining a set of customer sequences and, in addition, solving a MIQCP for each sequence. Since modern local-search and neighborhood-search methods evaluate a large number of candidate solutions, the efficient computation of visit points becomes a critical component of any practical algorithm for the CVRPN.

The CVRPN is NP-hard because it has the VRP as a special case. Therefore it is natural to address the CVRPN with a heuristic approach. In this paper, we consider a general variable neighborhood search (GVNS) method [33], where, given a current solution, different neighborhoods are inspected. Each neighborhood is formed by neighbors, and each neighbor corresponds to a set of routes. Now, building each route by determining its visit points so its length can be computed involves solving a problem of the form (1). Therefore, because we need solve problem (1) repeatedly, solving (1) efficiently is crucial for the overall approach to be practical.

The main contributions of this work are as follows. First, we introduce the capacitated vehicle routing problem with polygonal neighborhoods (CVRPN), a vehicle routing problem in which customers are represented by arbitrary polygonal regions, possibly non-convex. Second, we propose a mixed-integer quadratically constrained programming formulation for the problem. Third, we develop a block coordinate descent (BCD) procedure that efficiently computes visit points associated with a fixed sequence of customers. Fourth, we embed this route-evaluation procedure within a general variable neighborhood search (GVNS) metaheuristic specifically tailored to the CVRPN. Finally, we introduce benchmark instances and conduct extensive computational experiments that demonstrate the effectiveness of the proposed approach.

The rest of this paper is organized as follows. Section 2 describes the problem and presents a MIQCP model. Section 3 explains how to transform a visit sequence into a route. This is accomplished by computing the client visit points and determining the route’s length using block coordinate descent. Section 4 describes the GVNS method and the neighborhoods considered in this work. Section 5 presents exhaustive numerical experiments. Section 6 is devoted to conclusions and future work.

Notation. We denote by $\mathbb{R}_+$ the set of non-negative real numbers. For a point p in the plane, we denote its abscissa and ordinate by p^x and p^y , respectively. The symbol $\|\cdot\|$ represents the Euclidean norm. Given $(x, y) \in \mathbb{R}^2$, we denote $(x, y)^\perp = (y, -x)$, that is, $(x, y)^\perp$ corresponds to a 90-degree clockwise rotation of (x, y) . Given two points p and q on the plane, the segment joining them is denoted by $[p, q]$.

2 Problem description and model

The data for an instance of the CVRPN are: the position $p_0 \in \mathbb{R}^2$ of the depot, the maximum capacity $\mathcal{C} \in \mathbb{R}_+$, which is identical for all trucks in the fleet, the number of customers m and, for each customer $i \in \{1, \dots, m\}$, its demand $d_i \in \mathbb{R}_+$ and its polygonal neighborhood P_i . In this work, we consider polygonal neighborhoods given by polygons with at least three vertices. Without loss of generality, we assume that each polygon P_i is given by the union of m_i disjoint convex polygons, i.e. $P_i = \cup_{\ell=1}^{m_i} P_{i\ell}$, where the $P_{i\ell}$ are all convex. In other words, the collection

$\{P_{i\ell}\}_{\ell=1}^{m_i}$ makes up a partition of P_i into convex polygons. Furthermore, we assume¹ that each convex polygon $P_{i\ell}$ is expressed as $P_{i\ell} = \{p \in \mathbb{R}^2 \mid a_{i\ell k} p^x + b_{i\ell k} p^y + c_{i\ell k} \leq 0 \text{ for } k = 1, \dots, n_{i\ell}\}$, where $n_{i\ell}$ is the number of vertices of the polygon $P_{i\ell}$.

The main variables of the CVRPN are (a) the binary variables x_{ij} ($i, j = 0, \dots, m, i \neq j$) that indicate whether customer i precedes customer j on a route ($x_{ij} = 1$) or not ($x_{ij} = 0$), where customer 0 corresponds to the depot, and (b) continuous variables $p_i \in \mathbb{R}^2$ ($i = 1, \dots, m$) representing the point to be visited in the vicinity P_i of customer i . There are also continuous variables ξ_{ij} and γ_{ij} ($i, j = 0, \dots, m, i \neq j$) that help model the Euclidean distance between points p_i and p_j , continuous variables z_{ij} ($i, j = 0, \dots, m, i \neq j$) that help model capacity constraints and the elimination of subroutes, and binary variables $w_{i\ell}$ ($i = 1, \dots, m$ such that $m_i > 1, \ell = 1, \dots, m_i$) that help model $p_i \in P_i = \cup_{\ell=1}^{m_i} P_{i\ell}$.

The MIQCP model of the CVRPN follows. The model is based on the MILP model presented in [43] for the CVRP. This formulation serves two purposes. First, it provides a precise mathematical description of the problem addressed in this work. Second, it enables the computation of provably optimal solutions for small-scale instances using a commercial optimization solver. These instances, together with their provably optimal solutions, will later be used as part of the computational experiments to evaluate the effectiveness of the proposed solution method.

¹This is the assumption that requires polygons to have at least three vertices. It would be simple to include polygons with one and two vertices as well, but they would have to be represented differently. If a polygon Q is formed by a single vertex v , the constraint $q \in Q$ is written as $q = v$. If a polygon Q has exactly two vertices v_1 and v_2 , the constraint $q \in Q$ can be written as $q = v_1 + \lambda(v_2 - v_1)$ plus $0 \leq \lambda \leq 1$. These details could easily be added to the model that follows, but we prefer to ignore these cases to simplify the presentation.

$$\text{Minimize } \sum_{i=0}^m \sum_{\substack{j=0 \\ j \neq i}}^m \gamma_{ij} \quad (2)$$

$$\text{s.t. } x_{ij} \in \{0, 1\} \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (3)$$

$$\sum_{j=1}^m x_{0j} = \sum_{j=1}^m x_{j0} \quad (4)$$

$$\sum_{\substack{j=0 \\ j \neq i}}^m x_{ji} = 1 \quad i = 1, \dots, m \quad (5)$$

$$\sum_{\substack{j=0 \\ j \neq i}}^m x_{ij} = 1 \quad i = 1, \dots, m \quad (6)$$

$$z_{ij} \leq \mathcal{C}x_{ij} \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (7)$$

$$\sum_{\substack{\ell=0 \\ \ell \neq i}}^m z_{\ell i} - \sum_{\substack{\ell=0 \\ \ell \neq i}}^m z_{i\ell} = d_i \quad i = 1, \dots, m \quad (8)$$

$$\sum_{j=1}^m z_{0j} = \sum_{i=1}^m d_i \quad (9)$$

$$\sum_{i=1}^m z_{i0} = 0 \quad (10)$$

$$z_{ij} \geq 0 \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (11)$$

$$\xi_{ij}^2 \geq (p_i^x - p_j^x)^2 + (p_i^y - p_j^y)^2 \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (12)$$

$$\xi_{ij} \geq 0 \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (13)$$

$$\gamma_{ij} \leq Mx_{ij} \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (14)$$

$$\gamma_{ij} \geq \xi_{ij} - M(1 - x_{ij}) \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (15)$$

$$\gamma_{ij} \geq 0 \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (16)$$

$$a_{i\ell k}p_i^x + b_{i\ell k}p_i^y + c_{i\ell k} \leq 0 \quad i = 1, \dots, m \mid m_i = 1, \ell = 1, k = 1, \dots, n_{i1} \quad (17)$$

$$a_{i\ell k}p_i^x + b_{i\ell k}p_i^y + c_{i\ell k} \leq M(1 - w_{i\ell}) \quad i = 1, \dots, m \mid m_i > 1, \ell = 1, \dots, m_i, k = 1, \dots, n_{i\ell} \quad (18)$$

$$w_{i\ell} \in \{0, 1\} \quad i = 1, \dots, m \mid m_i > 1, \ell = 1, \dots, m_i \quad (19)$$

$$\sum_{\ell=1}^{m_i} w_{i\ell} = 1 \quad i = 1, \dots, m \mid m_i > 1 \quad (20)$$

The binary variables x_{ij} , whose domain is defined by the constraints (3), indicate whether customer i precedes customer j on a route ($x_{ij} = 1$) or not ($x_{ij} = 0$). Using these variables, constraint (4) states that the same number of vehicles that leave the depot must return to the depot, while constraints (5,6) state that each customer is visited exactly once. The continuous variables z_{ij} correspond to the quantity transported from customer i to customer j . Constraints (7) state that this quantity cannot exceed the maximum capacity of the truck and must be zero if customer i does not precede customer j on a route. Constraints (8) state that the quantity transported to customer i from its predecessor minus the quantity transported

from i to its successor must match the demand of customer i . Constraint (9) states that the sum of the quantities leaving the depot, in the direction of the first customer on each route, must match the total demand. Constraint (10) states that nothing should be transported from the last customer on each route back to the depot. Constraint (11) states that transported quantities must be non-negative. These constraints related to variables z_{ij} are responsible for the elimination of subroutes. So far, the constraints described are part of the model for the CVRP presented in [43]. Next, we describe the constraints that state that each visited point p_i must belong to the neighborhood P_i of customer i and the constraints that model the distances between consecutive points on a route.

Constraints (12,13) state that ξ_{ij} is an upper bound on the Euclidean distance between points p_i and p_j . For each pair of i and j , constraints (12,13) are equivalent to $\xi_{ij} \geq \|p_i - p_j\|_2$. This is a second-order cone (SOC) constraint, which defines a convex feasible set that can be handled by commercial exact solvers such as CPLEX or Gurobi. Although substituting the inequality by an equality in (12) would strengthen the model, it would also make it intractable for the aforementioned solvers. They will be used in numerical experiments to find optimal solutions for small instances as a way of evaluating the effectiveness of the proposed method. These are the constraints that make the model an MIQCP model instead of a plain MILP model. Note that the problem can be formulated as an MILP if the Manhattan or Chebyshev distances are considered instead of the Euclidean distance. Constraints (14,15,16) along with the objective function (2) ensure that, at optimal solutions, γ_{ij} coincides with ξ_{ij} if customer i precedes customer j in a route and is zero otherwise. Constraints (17) state that $p_i \in P_i$ and apply only to cases where P_i is convex, i.e., all i such that $m_i = 1$ and P_i is merely formed by the convex polygon P_{i1} . Constraints (18) state that $p_i \in P_{i\ell}$ if $w_{i\ell} = 1$ and are void if $w_{i\ell} = 0$. Constraints (19,20) state that the variables $w_{i\ell}$ are binary and that, for each fixed i , they must add up to 1. This implies that, when P_i is formed by the union of m_i disjoint convex polygons $P_{i\ell}$, p_i must belong to exactly one of them. The objective function (2) minimizes the sum of the distances between the visit points of consecutive customers on the routes, including the distances from the depot to the first and last visit points on each route.

In constraints (14,15,18), $M > 0$ is a sufficiently large constant. Consider that all polygons P_i are contained in a rectangle $[\underline{p}^x, \bar{p}^x] \times [\underline{p}^y, \bar{p}^y]$. In (14,15), it would be enough to take M as $\|(\bar{p}^x, \bar{p}^y) - (\underline{p}^x, \underline{p}^y)\|$, which is an upper bound on the distance between any two visit points. In (18), it would suffice to take M as the optimal value of

$$\text{Maximize}_{\{i=1,\dots,m|m_i>1,\ell=1,\dots,m_i,k=1,\dots,n_{i\ell}\}} \left\{ \text{Maximize}_{\{(p^x,p^y)^T \in [\underline{p}^x,\bar{p}^x] \times [\underline{p}^y,\bar{p}^y]\}} \{a_{i\ell k}p^x + b_{i\ell k}p^y + c_{i\ell k}\} \right\}.$$

We now attempt to strengthen model (2)–(20) by introducing bounds on the distances between pairs of visit points. We use the well-known norm inequalities $\|x\|_\infty \leq \|x\|_2 \leq \|x\|_1$ for all $x \in \mathbb{R}^n$. With the help of additional continuous variables ξ_{ij}^x and ξ_{ij}^y ($i, j = 0, \dots, m$, $i \neq j$),

these inequalities can be written as

$$-\xi_{ij}^x \leq p_i^x - p_j^x \leq \xi_{ij}^x \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (21)$$

$$-\xi_{ij}^y \leq p_i^y - p_j^y \leq \xi_{ij}^y \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (22)$$

$$\xi_{ij}^x, \xi_{ij}^y \geq 0 \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (23)$$

$$\xi_{ij} \geq \xi_{ij}^x \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (24)$$

$$\xi_{ij} \geq \xi_{ij}^y \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j \quad (25)$$

$$\xi_{ij} \leq \xi_{ij}^x + \xi_{ij}^y \quad i = 0, \dots, m, j = 0, \dots, m, i \neq j. \quad (26)$$

The practical value of these constraints will be evaluated through numerical experiments.

3 Computing visit points with block coordinate descent

Given a visit sequence, i.e., a sequence that starts at the depot, visits customers $i_1, i_2, \dots, i_\kappa$, and ends at the depot, the problem of finding the visit point for each of the customers on the sequence can be written as problem (1). The MIQCP model for this problem is analogous to model (2)–(20), but with variables x_{ij} and z_{ij} fixed and restricted only to the route at hand. As well as in (2)–(20), the optimal solution to this problem may be obtained using a commercial solver such as CPLEX or Gurobi. In this paper, we propose solving it using a block coordinate descent (BCD) method. To the best of our knowledge, block coordinate descent has not previously been exploited as a route-evaluation mechanism for vehicle routing problems with polygonal neighborhoods. What follows is a description of a version of BCD for problem (1). This version applies a cyclic selection of blocks, with each block corresponding to a visit point. For a more general version, which includes other ways of identifying blocks and selecting the block to be used in each iteration, see, for example, [7].

To simplify the presentation, we denote $\pi_j \equiv p_{i_j}$ and $\mathcal{P}_j \equiv P_{i_j}$ for $j = 1, \dots, \kappa$ and restate problem (1) as

$$\text{Minimize}_{\{\pi_j \in \mathcal{P}_j, j=1, \dots, \kappa\}} \Psi(\pi_1, \dots, \pi_\kappa) := \sum_{j=0}^{\kappa} \|\pi_{j+1} - \pi_j\|, \quad (27)$$

where, as in (1), π_0 and $\pi_{\kappa+1}$ refer to the known geographical position of the depot. A complete cycle of the BCD method consists of varying the index ℓ from 1 to κ and, for each ℓ , to solve

$$\pi_\ell = \operatorname{argmin}_{\{\pi \in \mathcal{P}_\ell\}} \{\|\pi_{\ell+1} - \pi\| + \|\pi - \pi_{\ell-1}\|\}. \quad (28)$$

Subproblem in (28) is derived from (27) by considering only π_ℓ as a variable while fixing all other π_j with $j \neq \ell$, and omitting the constant terms—those not depending on π_ℓ —in the objective function. The minimization variable is denoted by π , and π_ℓ is redefined as the optimal value of the subproblem. The type of subproblem that appears in (28) is very simple, contains only two variables, and its optimal solution can be efficiently computed. At the end of each cycle, we check whether a stopping criterion is satisfied. If the stopping criterion is satisfied, the visit points are given by π_j for $j = 1, \dots, \kappa$. If the stopping criterion is not satisfied, a new cycle is started. In practice, we consider two stopping criteria related to the maximum number of cycles

of the BCD method and lack of progress in the value of Ψ in consecutive iterations. We show in Section 3.1 how the subproblem in (28) is solved for a fixed ℓ .

Before proceeding, it is important to mention that the algorithm described above is a simplification of Algorithm 3.1 presented in [7]. In Algorithm 3.1 of [7], after solving (28), a decrease condition is tested. If this condition is not satisfied, one or more subproblems similar to (28), augmented by the regularization term $\sigma\|\pi - \pi_\ell\|^2$ for increasing values of the regularization parameter $\sigma > 0$, must be solved. These extra steps are essential for obtaining worst-case complexity results of the method and ensuring convergence to stationary points, as presented in [7]. Without these extra steps, there is a possibility that the method will cycle; see the discussion in [3]. However, solving subproblems of type (28) augmented with the regularized term is not as simple as solving (28). While the solution of (28) can be performed in a finite number of steps, the solution of its regularized variant would require a more expensive iterative method with asymptotic convergence. Considering that the cases in which the BCD method cycles are pathological and extremely rare in practice, and that we are using BCD within the context of a heuristic method, we believe that the basic version used here is the most appropriate option. The main purpose of the proposed BCD procedure is not to guarantee global optimality of the visit-point problem, but rather to provide high-quality route evaluations at a computational cost compatible with large-scale neighborhood search.

3.1 Solving the BCD subproblems

In this section, we show how to solve problem (28) for a fixed $\ell \in \{1, \dots, \kappa\}$. To simplify the presentation, we restate the problem as

$$\underset{z \in \mathcal{Q}}{\text{Minimize}} \phi(z) := \|p - z\| + \|z - q\|, \quad (29)$$

where $p, q \in \mathbb{R}^2$ and $\mathcal{Q}$ is a non-convex polygon. By the triangle inequality, the optimal value of ϕ is greater than or equal to $\|p - q\|$. If $[p, q] \cap \mathcal{Q} \neq \emptyset$, then any point $z^* \in [p, q] \cap \mathcal{Q}$ achieves the optimal value and is therefore a solution. We now intend to show that, when $[p, q] \cap \mathcal{Q} = \emptyset$, a solution z^* is necessarily on the boundary of $\mathcal{Q}$. Let z be in the interior of $\mathcal{Q}$. Since $[p, q] \cap \mathcal{Q} = \emptyset$, p , q , and z are not collinear and thus form a triangle $\triangle pqz$. Let w be a point in the segment $[p, q]$ and let d be a direction leaving z in the direction of w ; see Figure 2. It is easy to see that d is feasible, since z is in the interior of $\mathcal{Q}$, and simultaneously reduces the length of the sides $[p, z]$ and $[q, z]$. That is, there exists $\bar{\epsilon} > 0$ such that $z + \epsilon d \in \mathcal{Q}$ and $\phi(z + \epsilon d) < \phi(z)$ for all $\epsilon \in (0, \bar{\epsilon}]$. Therefore, z does not satisfy the definition of local minimizer and cannot, thus, be a solution to (29). We conclude that, to solve (29) when $[p, q] \cap \mathcal{Q} = \emptyset$, we need only to search for a solution on the boundary of $\mathcal{Q}$. The above argument does not depend at all on $\mathcal{Q}$ being a polygon and remains valid for a neighborhood given by any compact set. Next, we describe how to find a solution on the boundary of $\mathcal{Q}$. This description depends heavily on the fact that $\mathcal{Q}$ is a non-convex polygon.

We now outline how to compute a solution to (29) on the boundary of $\mathcal{Q}$ when $[p, q] \cap \mathcal{Q} = \emptyset$. Recall we are assuming that $\mathcal{Q}$ has $h \geq 3$ vertices² that here we name v_ℓ ($\ell = 1, \dots, h$) in

²Solving (29) when $\mathcal{Q}$ has a single vertex is trivial; while the case with two vertices is similar to the case with three or more vertices, but only two vertices and one edge need to be verified. Treating the case with two vertices in the same way as situations involving three or more vertices would result in the same edge being tested twice.

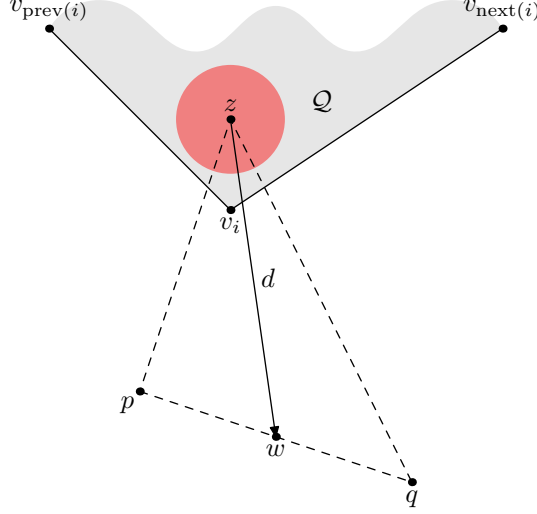


Figure 2: This figure illustrates that a point z in the interior of $\mathcal{Q}$ cannot be a solution of (29) when the segment $[p, q]$ has no intersection with the polygon $\mathcal{Q}$. This fact follows from the existence of a direction d leaving z that is simultaneously feasible and a descent direction.

counterclockwise order. To simplify the presentation, given $\ell \in \{1, \dots, h\}$, we define $\text{next}(\ell) = \ell + 1$ whenever $\ell < h$ and $\text{next}(h) = 1$. Analogously, we define $\text{prev}(\ell) = \ell - 1$ whenever $\ell > 1$ and $\text{prev}(1) = h$. Because we limit the search to the boundary of $\mathcal{Q}$, we can parameterize its boundary with a single scalar parameter λ and rewrite (29) as

$$\underset{\lambda \in \mathbb{R}}{\text{Minimize}} \psi(\lambda) := \|p - z(\lambda)\| + \|z(\lambda) - q\| \text{ subject to } 1 \leq \lambda \leq h + 1, \quad (30)$$

where, for $i = 1, \dots, h$, if $\lambda \in [i, i + 1]$, then $z(\lambda) := v_i + (\lambda - i)(v_{\text{next}(i)} - v_i)$. Note that if $\lambda = i$ then $z(\lambda) = v_i$ for $i = 1, \dots, h$ and that non-integer values of λ correspond to points in the interior of the edges of $\mathcal{Q}$. Concretely, if $\lambda \in (i, i + 1)$, then $z(\lambda) \in (v_i, v_{\text{next}(i)})$. The key to finding a solution to (30) is that the function $\psi(\lambda)$ is strictly convex (sum of norms of affine functions) within each interval $\lambda \in [i, i + 1]$. The fact that $[p, q] \cap \mathcal{Q} = \emptyset$ guarantees that ψ is differentiable in $(i, i + 1)$. See Figure 3. Therefore, $z(\lambda) \in [v_i, v_{\text{next}(i)}]$ can be a solution only if $\lambda \in [i, i + 1]$ is such that $\lambda = i$, $\lambda = i + 1$ or $\lambda \in (i, i + 1)$ and $\psi'(\lambda) = 0$. In other words, the candidate solutions are the vertices of $\mathcal{Q}$ and the points in the interior of the edges that nullify the derivative of ψ . Among these candidates, the one with the lowest value of ψ is the desired solution.

For $i = 1, \dots, h$, if $\lambda \in (i, i + 1)$, the derivative of ψ is given by

$$\psi'(\lambda) = \left\langle -\frac{p - z(\lambda)}{\|p - z(\lambda)\|} + \frac{z(\lambda) - q}{\|z(\lambda) - q\|}, v_{\text{next}(i)} - v_i \right\rangle; \quad (31)$$

and therefore $\psi'(\lambda) = 0$ if and only if

$$\frac{\langle z(\lambda) - q, v_{\text{next}(i)} - v_i \rangle}{\|z(\lambda) - q\|} = \frac{\langle p - z(\lambda), v_{\text{next}(i)} - v_i \rangle}{\|p - z(\lambda)\|}. \quad (32)$$

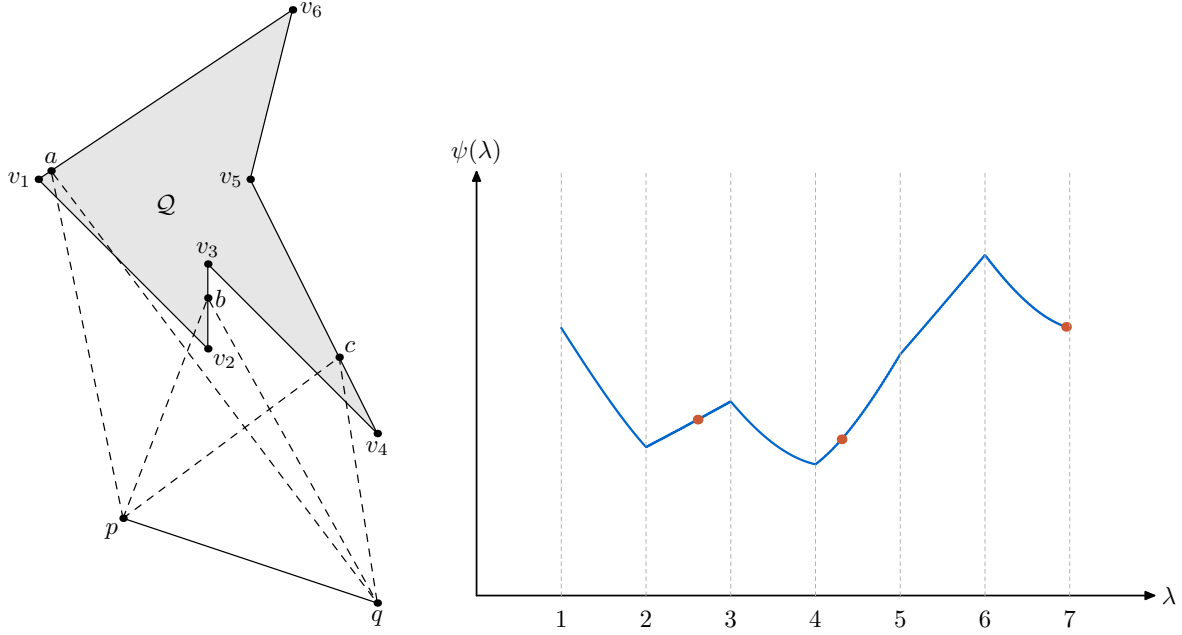


Figure 3: (a) Graphical representation of the parameterization of the boundary of a non-convex polygon $\mathcal{Q}$ with vertices $v_1, v_2, \dots, v_6$ in counterclockwise order. The whole boundary corresponds to varying λ between 1 and 7. The point $a = v_6 + 0.95(v_1 - v_6)$ corresponds to $\lambda = 6.95$; the point $b = v_2 + 0.6(v_3 - v_2)$ corresponds to $\lambda = 2.6$; the point $c = v_4 + 0.3(v_5 - v_4)$ corresponds to $\lambda = 4.3$. In this example, $[p, q] \cap \mathcal{Q} = \emptyset$. (b) Graph of $\psi(\lambda)$. The graph highlights in red the pairs $(\lambda, \psi(\lambda))$ that correspond to points a , b , and c . It is clear that the function is convex and differentiable between integer values of λ and non-differentiable for integer values of λ . A simple inspection of both figures shows that $z^* = z(4) = v_4$.

By squaring both sides (to eliminate the square root) and performing the cross product, we obtain a quartic equation, which reduces to a quadratic equation. Among the two roots of the quadratic equation, the one that satisfies (32) corresponds to the minimizer of ψ on the line passing through the vertices v_i and $v_{\text{next}(i)}$. Let us call it λ_i^* . If $\lambda_i^* \in (i, i+1)$, then $z(\lambda_i^*) \in (v_i, v_{\text{next}(i)})$ and $z(\lambda_i^*)$ is a candidate solution to (29). Let $\mathcal{S}$ be the set of candidate $z(\lambda_i^*)$ plus the vertices of $\mathcal{Q}$. The solution z^* to (29) is given by $z^* = \operatorname{argmin}_{\{z \in \mathcal{S}\}} \{\phi(z)\}$.

4 General variable neighborhood search

General variable neighborhood search (GVNS) is a variation of the variable neighborhood search (VNS) [22, 23, 33] metaheuristic designed to enhance exploration and intensification in the search process. VNS systematically explores different neighborhoods to escape local optima by alternating between a shaking phase (to diversify) and a local search phase (to improve the solution). However, it typically uses a single neighborhood structure in each local search step. GVNS strengthens this approach by incorporating a variable neighborhood descent (VND) within the

local search phase. The VND method applies several neighborhood structures sequentially. This deeper more exhaustive search yields a better-quality solution before returning to the shaking step. This hierarchical structure makes GVNS more robust and effective than the basic VNS, especially for complex combinatorial problems, such as the VRP and its variants.

When applied to VRP variants, GVNS has delivered strong performance. Recent works [6, 14, 26, 31, 34, 38, 39] show that GVNS can achieve near-optimal or optimal solutions for many VRP benchmark instances, performing competitively against leading metaheuristics such as Tabu Search, Genetic Algorithms, and Ant Colony Optimization. The VRP’s structure allows, in a natural fashion, for multiple neighborhood definitions, primarily based on route swaps and customer relocations. This variety of neighborhood definitions fit perfectly within the GVNS framework, enabling it to effectively navigate the VRP’s complex solution space. In this study, we considered the intra-route neighborhoods **(i)** 2-Opt, **(ii)** 3-Opt, and **(iii)** Or-Opt and the inter-route neighborhoods **(iv)** 1-0-Exchange, **(v)** 1-1-Exchange, **(vi)** 1-2-Exchange, **(vii)** Relocate, **(viii)** Cross-Exchange, and **(ix)** 2-Opt-Star; see [1, 40].

In the following, we describe implementation details of each of the considered neighbourhoods. We implemented the 2-Opt neighborhood as described in the literature. The 3-Opt neighborhood, as described in the literature, includes the movements of 2-Opt. In our implementation, as we intend to use both combined, we implemented in 3-Opt only those movements that are not 2-Opt. The Or-Opt neighborhood, given a route, considers segments with r consecutive customers, with r varying from $r_{\min}$ to $r_{\max}$, and reallocates the segment in all possible positions on the route. Unlike the classic version described in the literature, when reallocating, we also consider reallocating the inverted sequence. We implemented the 1-0-Exchange and the 1-1-Exchange neighborhoods exactly as described in the literature. The 1-2-Exchange considers two routes; from one it takes a segment with a single customer, from the other it takes a segment with two consecutive customers, and exchanges these segments on the routes. Unlike the classic version described in the literature, when inserting the segment with two customers, we consider two possibilities: the usual insertion and the insertion with the inverted segment. The Relocate neighborhood is an extension of 1-0-Exchange. It takes segments with r consecutive customers from one route, with r varying from $r_{\min}$ to $r_{\max}$, removes them from the route, and inserts them into another route. Unlike the classic version described in the literature, when inserting, we also consider the inverted sequence insertion. The Cross-Exchange neighborhood is an extension of 1-1-Exchange. It considers two routes and exchanges consecutive segments with r_1 and r_2 customers from each route. Unlike the classic version described in the literature, we consider the four combinations in which the segments are reinserted, inverted or not. In the experiments, we consider all pairs (r_1, r_2) in a predefined set of pairs R . The pairs $(1, 0)$ and $(0, 1)$ would generate the movements of the 1-0-Exchange neighborhood. Other pairs containing 0, either as the first or second coordinate, would generate movements in the Relocate neighborhood. The pair $(1, 1)$ would generate movements in the 1-1-Exchange neighborhood, and the pairs $(1, 2)$ and $(2, 1)$ would generate movements in the 1-2-Exchange neighborhood. Therefore, the set R can be defined as $R = \{(r_1, r_2) \mid r_{\min} \leq r_1, r_2 \leq r_{\max}\} \setminus \{(1, 1), (1, 2), (2, 1)\}$, with $r_{\max} \geq r_{\min} \geq 1$. We implemented 2-Opt-Star exactly as described in the literature.

The natural value for the parameter $r_{\min}$ in the Or-Opt and Cross-Exchange neighborhoods is $r_{\min} = 1$. In the case of the Relocate neighborhood, the natural value is $r_{\min} = 2$, since with $r = 1$ the movements are those already generated by the 1-0-Exchange neighborhood.

There are no strong reasons for the three neighborhoods to use different values for $r_{\max} \geq 2$. Therefore, we will consider a unified value for all three. This value will be the subject of numerical experimentation. During the parameter calibration phase, we will also evaluate the inclusion of movements with inverted sequences. These movements will be prohibited if `FLIP = false` and allowed if `FLIP = true`. The decision on whether to use one or the other will be the subject of numerical experimentation. Related to the use of neighborhoods, as is usual in local searches, we will analyze whether we should use the first-improvement strategy and abandon the construction of the neighborhood when a neighbor better than the current solution is found, or whether the inspection of the neighborhood should be complete in order to choose the best neighbor as the new solution. We will consider the parameter `OTYPE` during the calibration phase, assigning value `F` to the first-improvement strategy and value `B` to the best-improvement strategy.

Our implementation of GVNS corresponds precisely to Algorithms 4, 5, and 8 presented on pages 372, 373, and 376 of [23]. It is interesting to note that the method has no parameters other than those related to the neighborhoods. The neighborhoods considered are the nine neighborhoods mentioned above. There is a shake associated with each neighborhood, and the shake consists of making a random movement of the associated neighborhood. However, preliminary experiments have shown that a single movement of a single neighborhood does not provide the necessary diversification to the method. We therefore consider a shake in which we first randomly select a quantity q between 1 and $q_{\max}$ and then perform q random movements of q random neighborhoods. The quantity $q_{\max}$ of consecutive random movements that make up the shake will be subject to numerical calibration. Another factor relevant to the performance of the method is the order in which the neighborhoods are considered in the VND. In the numerical experiments, we consider four basic possibilities. The first is to consider the neighborhoods in the order in which they were described. The second possibility is to consider first the six inter-route neighborhoods (in the order in which they were described) and then the three intra-route neighborhoods (also in the order in which they were described). In these two options, the order is maintained throughout the entire execution of the method. The other two possibilities consist of starting with one of the two orders already mentioned and changing the order to a random order each time the method completes a cycle of nine shakes without improvement of the incumbent. The choice of this parameter, which we will refer to as `NEIGHBORHOOD_ORDER` $\in \{1, 2, 3, 4\}$, will be subject to calibration.

5 Numerical experiments

In this section, we present numerical experiments to evaluate the performance of the proposed metaheuristic enhanced with block coordinate descent. Different aspects of the proposed model and method will be individually evaluated. Section 5.1 describes two simple constructive heuristics that deliver a solution to be used as initial solution for either the GVNS metaheuristic or an exact commercial solver. In Section 5.2, we introduce small- and medium-scale CVRPN instances, each with 3 to 60 customers. These instances are used, in Section 5.3, to evaluate the inclusion of the redundant constraints (21)–(26), based on well-known norm inequalities, in the MICQP model (2)–(20). The same instances are used, in Section 5.4, to compare two forms of calculating the visit points: using BCD versus solving a MICQP model with the commercial

solver CPLEX. The evaluation focuses on the effectiveness and efficiency aspects of BCD. In Section 5.5, we rely on instances from the capacitated VRP literature to construct large-scale instances of the CVRPN with polygonal neighborhoods. A small subset of the large-scale instances is used in Section 5.6 to calibrate the parameters of the proposed GVNS metaheuristic. Sections 5.7 and 5.8 describe the performance of the GVNS metaheuristic in the small-, medium-, and large-scale instances of the CVRPN.

The GVNS metaheuristic was implemented in Fortran 90. To facilitate reproducibility, the complete source code, all generated benchmark instances, and the solutions reported in this paper are publicly available at <http://www.ime.usp.br/~egbirgin/>. The experiments were carried out in an Intel i9-12900K (12th Gen) processor operating at 5.200GHz and 128 GB of RAM, running Ubuntu 23.04. The code was compiled by the GNU Fortran compiler GCC (version 12.3.0) with the -O3 optimization directive enabled. Models were solved using IBM ILOG CPLEX Optimization Studio 22.1.2 with all its default parameters.

5.1 Heuristics to construct feasible solutions

To both initialize the GVNS metaheuristic and assist in solving the model through the application of an exact method, it is useful to have a constructive heuristic that can generate an initial feasible solution. In this work, we consider simple adaptations of the constructive heuristics proposed by Clarke and Wright [11] (saving method) and by Flood [18] (nearest neighbor) for the VRP. In both cases, the heuristics are applied to a CVRP instance derived from the corresponding CVRPN instance. This derived instance is built by assigning to each customer the barycenter of its polygonal neighborhood as its geographical position. We check the capacity constraint before applying each heuristic’s main step: merging two routes (saving method), or adding a new customer to an existing route (nearest neighbor). The application of either constructive heuristic yields a set of visit sequences. By applying BCD to each sequence, we determine the visit points and obtain a complete set of routes.

5.2 Generation of small- and medium-scale instances of the CVRPN

We generated 134 instances using the scheme proposed in [28]. The positions of the customers correspond to points that were randomly selected within the square $[-1, 1]^2$. However, rather than randomly selecting a single geographic position for each customer c , we first randomly selected a scalar number, say m_c . We then randomly selected a cluster of m_c points that will be associated with customer c . All draws follow a uniform distribution. The generation of points for each cluster considers a random reference point and two squares centered on the reference point with sides of length $2r_{\min}$ and $2r_{\max}$, respectively, where $r_{\min}$ and $r_{\max}$ are constants that satisfy $0 < r_{\min} < r_{\max} \ll 1$. The cluster points are drawn in such a way that they are outside the smaller square and inside the larger square. Also, points within the same cluster are at a distance $d_{\min} > 0$ or greater from each other, measured in the Euclidean norm. To achieve this, points are drawn within the larger square, discarding those that fall within the smaller square or at a distance of less than $d_{\min}$ from a previously drawn point in the same cluster. See Figure 4. The reference point is drawn in $[-1 + r_{\max}, 1 - r_{\max}]^2$, so that the entire cluster is contained within $[-1, 1]^2$. If the drawn reference point is at a distance (in the infinite norm) of less than

$2r_{\max}$ from a reference point of a previous customer, it is discarded, and a new reference point is drawn. The result of this process is a set of “chunky clusters without overlap”. The depot is always located at the origin.

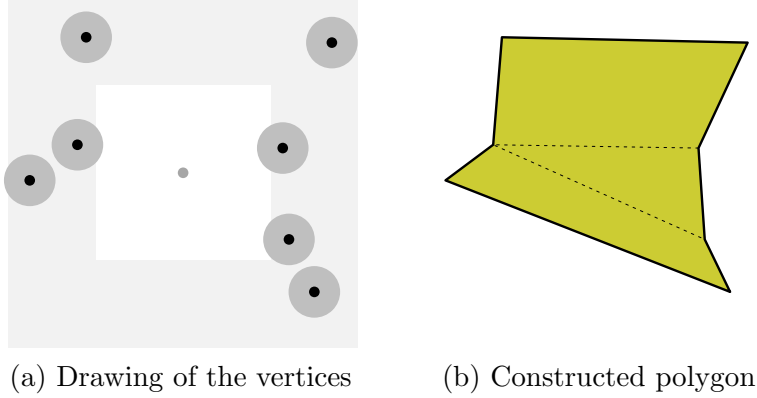


Figure 4: This figure illustrates the construction of a non-convex polygonal neighborhood. Figure (a) shows a random reference point (light gray) and two squares with sides $2r_{\min}$ and $2r_{\max}$ centered on the reference point. It also shows the $m_c = 7$ points drawn at random within the larger square and outside the smaller square, with minimum distance $d_{\min}$. Figure (b) shows the polygon constructed with the drawn points and its division into convex subpolygons. The division is used only in the context of the MIQCP model.

Up to this point, the problem description could be considered a generalized vehicle routing problem (GVRP) instance. By representing in polar coordinates the points associated with a customer, and ordering them from smallest to largest by angle, we obtain vertices (ordered in a counterclockwise direction) of a non-convex polygon representing the customer’s region. In order for the instance to be described with model (2)–(20), each non-convex polygon needs to be partitioned into a finite number of convex polygons. We did this using the subroutine `optimal_convex_partition_2` from the CGAL library [25]. For each customer, we randomly selected a demand d_i from the set $\{1, 2, \dots, 10\}$. We generated instances with the number of customers m varying from three to 60.

For instances with up to 15 customers, we set a maximum vehicle capacity of $\mathcal{C} = 30$ and the number of points per cluster between 3 and 7. For instances with more than 15 customers, we set a maximum capacity of $\mathcal{C} = 50$ and the number of points per cluster between 3 and 12. For instances with $m = 3$ customers, we set $r_{\min} = 0.1$ and $r_{\max} = 0.2$, and for instances with $3 < m \leq 15$, we set $r_{\min} = 0.1\sqrt{3/m}$ and $r_{\max} = 0.2\sqrt{3/m}$. Similarly, for instances with $m = 16$, we set $r_{\min} = 0.05$ and $r_{\max} = 0.1$ and, for instances with $16 < m \leq 60$, we set $r_{\min} = 0.05\sqrt{16/m}$ and $r_{\max} = 0.1\sqrt{16/m}$. In all instances, we set $d_{\min} = 0.4\sqrt{16(r_{\max}^2 - r_{\min}^2)/(\pi m_c)}$. The total number of vertices of the polygons that constitute an instance varies between 9 and 473. The total number of convex polygons that compose the partitions of the non-convex polygonal regions varies from 3 to 140. When applied to the instances, model (2)–(20) has up to 3,784 binary variables, 11,100 continuous variables, 11,840 linear constraints, and 3,660 quadratic constraints.

These quantities do not include the additional variables and constraints from (21)–(26). This set of additional constraints does not add binary variables or quadratic constraints. It adds only $2(m+1)m$ continuous variables and $7(m+1)m$ linear constraints for an instance with m customers. Tables 1 and 2 in [8] show details of the generated instances.

5.3 Evaluation of the effect of the MICPQ model’s additional constraints

We tried to solve model (2)–(20) using CPLEX. We used CPLEX with its default parameters. We provided as an initial solution the best among the solutions constructed with the saving method and the nearest neighbor heuristic adapted as mentioned in the Section 5.1. We set a CPU time limit of 1 hour. Details of the results obtained with CPLEX applied to model (2)–(20), including and not including the additional constraints (21)–(26), are shown in Tables 3 and 4 in [8].

To analyze the results, we divided the 134 instances into two sets. Set A consists of 70 instances in which CPLEX closed the optimality gap when applied to model (2)–(20) in both ways, with and without the additional constraints (21)–(26); while set B consists of the remaining 64 instances. In set A, we used CPU time as a measure of efficiency. After confirming with a Shapiro-Wilk test [36] that the data did not follow a normal distribution, we performed a nonparametric Wilcoxon test [44] at a significance level of $\alpha = 0.05$ to accept or reject the null hypothesis that the CPU time samples from the two methods come from the same distribution. The p -value equal to 0.41 shows that the null hypothesis should be accepted, that is, that the additional constraints do not interfere with the CPU time used by CPLEX to find a guaranteed optimal solution to model (2)–(20). In set B, we consider the best solution found as a measure of effectiveness. The Shapiro-Wilk that confirmed the data did not follow a normal distribution and the Wilcoxon test yielded a p -value close to 0.001, showing that the solutions found using and not using the additional constraints do not come from the same distribution. The use of the additional constraints leads to better quality solutions in 25 instances, lower quality solutions in 4 instances, and solutions of equal quality in the remaining 35 instances. When the Wilcoxon test is applied to the gaps obtained by CPLEX for the instances in set B, the p -value of the order of 10^{-5} shows that the additional constraints also have a significant influence on the final gaps, obtained within the limit of one hour of CPU time. The use of the additional constraints leads to smaller gaps in 49 instances and larger gaps in the remaining 15 instances. Overall, it is clear that the use of the additional constraints improves the effectiveness of the commercial CPLEX solver.

5.4 Evaluation of using BCD to compute the visit points of a visit sequence

Given a visit sequence, i.e., a sequence that starts at the depot, visits customers $i_1, \dots, i_\kappa$, and ends at the depot, we propose calculating the visit points using BCD, as described in Section 3. Alternatively, the visit points can be calculated by solving a simplified version of the MIQCP model (2)–(26), which corresponds to fixing the binary variables that define the visit sequence. In this section, we aim to show that using BCD is several orders of magnitude more efficient, to the point that without its use, the application of the GVNS metaheuristic proposed in this work would be impractical. For the experiment, we considered the 134 instances introduced in

Section 5.2. For each of them, we considered the best set of visit sequences (set with the lowest sum of lengths) among the solutions calculated with the constructive heuristics described in Section 5.1. For these visit sequences, we computed the visit points in two ways: by solving the simplified version of the MIQCP model (2)–(26) and by using BCD. Tables 5 and 6 in [8] show the details.

The analysis of the results shows that, in terms of efficiency, BCD is three to eight orders of magnitude faster than CPLEX. While BCD uses a CPU time of around 10^{-5} seconds, CPLEX takes between one hundredth of a second and one hour, depending on the instance. There are even four instances in which CPLEX was unable to find a guaranteed optimal solution within the one-hour CPU time limit. Of course, the comparison of times should be taken with caution, since we are comparing an exact method with a heuristic method. In terms of the quality of the solution found, CPLEX and BCD find solutions of equivalent quality in 125 of the 134 instances. The remaining nine instances, in which CPLEX finds a slightly worse solution, are divided into two groups. In six of them, the difference of the order of 0.001 is due to CPLEX’s stopping criterion, which stops when the absolute or relative gaps are below a certain threshold. The remaining three instances, in which the difference is slightly greater, correspond to cases in which CPLEX stopped after reaching the time limit. In summary, BCD always found solutions of equal or better quality than those found by CPLEX and was significantly more efficient.

5.5 Large-scale instances derived from literature CVRP instances

We consider the CVRP instances available in the CVRPLib library (<https://galgos.inf.puc-rio.br/cvrplib/>, accessed June 15, 2026). The steps for converting a CVRP instance to a CVRPN instance are as follows. First, for each customer, we randomly select a number of points nv , between $nv_{\min} = 3$ and $nv_{\max} = 8$. Then, we draw a cluster of nv points close to the customer’s geographic coordinates. We express the cluster points in polar coordinates and sort them in ascending order by angle. The points then represent the vertices of a potentially non-convex polygonal neighborhood. We then check if the customer belongs to its neighborhood. If not, we replace a random point in the cluster with the customer, reorder the cluster points by angle, and obtain a new neighborhood that is guaranteed to include the customer. The conversion process preserves the location of the depot. As a result, any solution to the original CVRP instance (in particular, the optimal or best solution reported in the literature) is a feasible solution to the generated CVRPN instance. Consequently, the best known solution for the CVRP instance can serve as an upper bound for the optimal solution of the CVRPN instance, which we aim to either achieve or improve upon in our numerical experiments.

Using the procedure described above, we generated CVRPN instances based on the A, B, E, F, M, P, and Tai benchmark sets. In total, we generated 106 CVRPN instances. Across these instances, the number of customers ranges from 15 to 385, while the total number of neighborhood vertices ranges from 86 to 2,088. Tables 7 and 8 in [8] show details of the generated instances. For the largest generated instance, model (2)–(26) comprises nearly 150,000 binary variables, 650,000 continuous variables, 1.5 million linear constraints, and 150,000 quadratic constraints.

5.6 Calibration of the parameters of the GVNS metaheuristic

We automatically calibrated the parameters of the GVNS heuristic using Irace [30]. We set $maxExperiments = 1000$ and kept all remaining parameters at their default values. Initially, we attempted to perform the calibration using the 134 small- and medium-scale instances described in Section 5.2. However, some of these instances appeared to be too easy, in the sense that the GVNS metaheuristic was able to find an optimal solution for almost any parameter combination, which impaired the parameter selection process conducted by Irace. At the same time, preliminary experiments indicated that the set of 13 instances adapted from the Tai set of the CVRPLib Library were considerably more challenging. Therefore, we decided to use them for parameter calibration. The search space included the parameters $OTYPE \in \{B, F\}$, $r_{max} \in \{2, \dots, 10\}$, $NEIGHBORHOOD_ORDER \in \{1, \dots, 4\}$, $q_{max} \in \{1, \dots, 5\}$, and $FLIP \in \{true, false\}$. We adopted a time limit of 1200 seconds per run. The best configuration returned by Irace was $OTYPE = F$, $r_{max} = 5$, $NEIGHBORHOOD_ORDER = 4$, $q_{max} = 3$, and $FLIP = false$. Irace identified $OTYPE = F$, $r_{max} = 10$, $NEIGHBORHOOD_ORDER = 3$, $q_{max} = 5$, and $FLIP = false$ as the second-best parameter configuration, and $OTYPE = B$, $r_{max} = 5$, $NEIGHBORHOOD_ORDER = 1$, $q_{max} = 4$, and $FLIP = true$ as the third-best configuration. Since our GVNS implementation underwent several minor modifications during its development, we repeated the calibration procedure multiple times. Across all runs, Irace consistently selected the first-improvement strategy ($OTYPE = F$) together with $FLIP = false$ as part of the best-performing parameter configuration. Disabling the flip operation reduces the neighborhood size, while the first-improvement strategy typically avoids exploring the entire neighborhood, allowing the method to perform more iterations within the prescribed time limit. In contrast, the remaining parameters varied across different calibration runs, suggesting that the GVNS metaheuristic is relatively robust with respect to those parameter choices.

5.7 Experiments with the small and medium-scale instances of the CVRPN

We ran the GVNS metaheuristic on the 134 small- and medium-scale instances described in Section 5.2. We adopted the parameter configuration obtained with Irace and imposed a time limit of 1200 seconds per run. Since the GVNS heuristic contains randomized components, each instance was solved 10 times. Tables 9 and 10 in [8] report the best result obtained over the 10 runs for each instance. These results can be compared with the solutions obtained by CPLEX, using a time limit of one hour. For each instance, we considered the best solution obtained by CPLEX over models (2)–(20) and (2)–(26). Considering both models, CPLEX found a provably optimal solution for 73 instances, which essentially correspond to the smallest instances in the test set. In all such cases, the GVNS heuristic also found an optimal solution. For the remaining 61 instances, CPLEX reached the time limit without proving optimality. Among these 61 instances, the GVNS heuristic matched the best solution value obtained by CPLEX (considering both models) in 15 cases. In the remaining 46 instances, the solution found by GVNS improved upon the best solution obtained by CPLEX. On average, these solutions were 2.96% better. Overall, the GVNS metaheuristic matched or improved all solutions obtained by CPLEX when considering the best results over both mathematical models.

5.8 Experiments with the large-scale instances adapted from the CVRP literature

We ran the GVNS metaheuristic on the 106 large-scale instances described in Section 5.5. We adopted the parameter configuration obtained with Irace and imposed a time limit of 1200 seconds per run (more on this later). Since the GVNS heuristic contains randomized components, each instance was solved 10 times. Tables 11 and 12 in [8] summarize the computational results. For each instance, we report the best total distance obtained across the 10 runs, the average total distance over all runs, and the coefficient of variation (defined as the standard deviation divided by the average total distance). In addition to the total distance statistics, we also report performance metrics. For 52 of the 106 instances, corresponding to the smaller instances in each set, the best solution is found in all 10 runs. For the remaining 54 instances, the frequency with which the best solution is found over the 10 runs decreases as the number of customers increases. For these 54 instances, the coefficient of variation never exceeds 1.41% and averages 0.27%. As an illustration, Figure 5 shows the solution obtained for instance P-n101-k4 from benchmark set P.

To validate the performance of the proposed GVNS, we solved the 106 large-scale instances using CPLEX; these results are reported in Tables 13 and 14 in [8]. CPLEX achieved optimality in 3 instances and found feasible solutions for 102. In one specific instance (Tai385), it failed to return a feasible solution due to an out-of-memory error. GVNS matched all 3 optimal solutions. Among the remaining 102 instances, GVNS tied with CPLEX in 6 and outperformed it in 96, yielding an average improvement of 4.58%.

To strengthen the performance validation, for each CVRPN instance, we also compared the solution obtained by GVNS with the best-known solution reported in the literature for the corresponding CVRP instance from which the CVRPN instance was derived. Since the best-known solution of the original CVRP instance is also feasible for the associated CVRPN instance, it provides a valid upper bound. The results show that the solutions computed by GVNS improved upon this upper bound in all instances, with an average improvement of 5.34%. When a CVRP instance is transformed into a CVRPN instance, the geographical location of each customer is replaced by a neighborhood containing that location. This modification enlarges the feasible region of the problem. A natural question is therefore whether the observed improvement arises merely from relocating the visit points associated with an optimal CVRP solution while preserving its routing structure. For only 8 out of the 106 tested instances the CVRPN solution exhibited the same routes and customer visitation sequences as the corresponding CVRP solutions reported in the literature. In the remaining 98 instances, the enlarged feasible region led to qualitatively different solutions, involving changes not only in the selection of customer visit points within their neighborhoods but also in the routing structure itself. These results indicate that the additional flexibility introduced by neighborhoods fundamentally alters the nature of the optimal solution rather than simply refining the geographical placement of visits. The fact that GVNS improves upon these upper bounds is particularly noteworthy because, for sets A, B, E, F, M, and P, all reported CVRP solutions are proven optimal. For the Tai set, all solutions are proven optimal except for instance Tai385.

The 1200-second time limit adopted in the computational experiments of this section was chosen arbitrarily. Since it is unclear whether GVNS could find better-quality solutions with

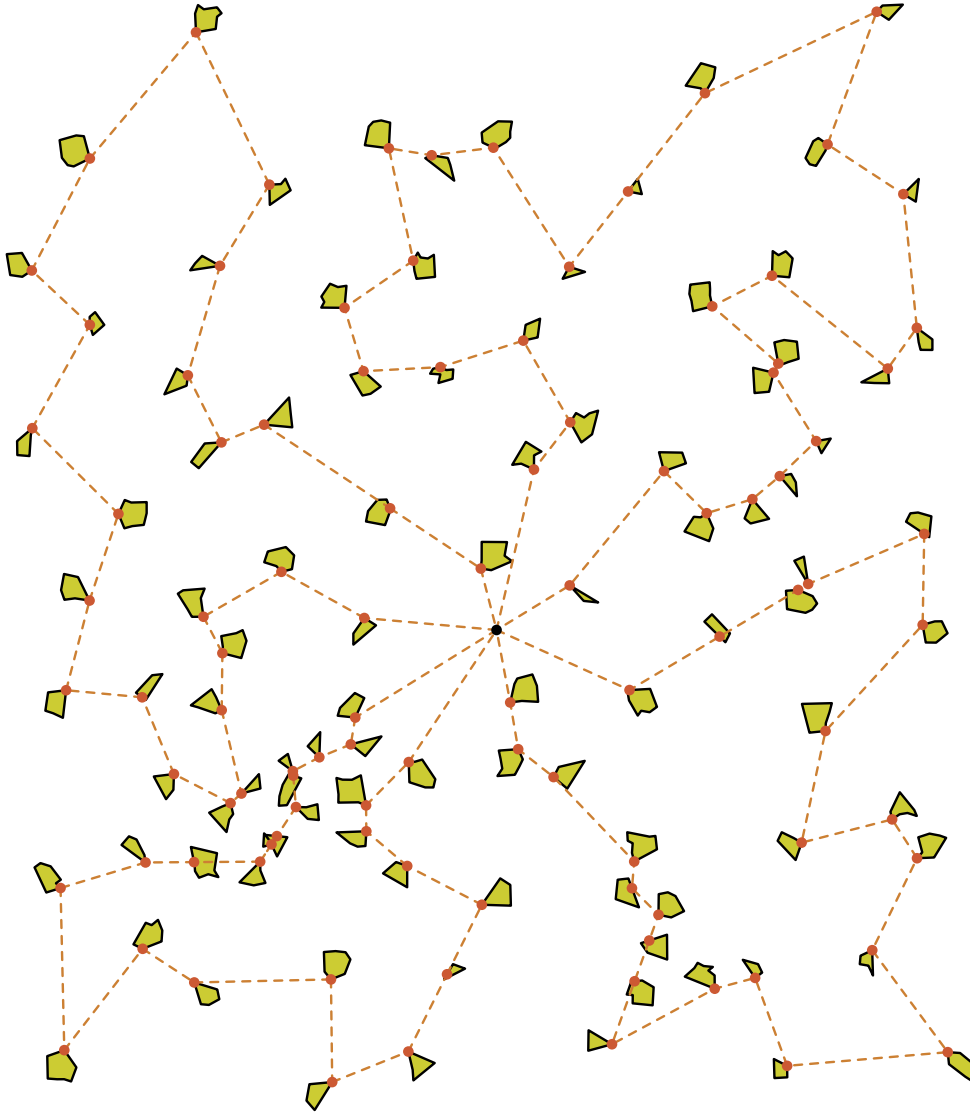


Figure 5: Graphical representation of the solution obtained for instance P-n101-k4. The solution consists of four routes: three serving 24 customers each and one serving 28 customers, with a total distance of 611.03. Although the solution may appear amenable to straightforward improvements, we ask the reader to bear in mind that the vehicle-capacity constraints severely restrict the feasible reassignment of customers among routes.

additional computational time and whether this limit influenced Irace’s selection of the best parameter configuration, we reran GVNS on all 106 instances using the three best parameter configurations identified by Irace, this time with a time limit of five hours. Because this experiment was computationally expensive, each instance was solved only once. Tables 15 and 16 in [8] summarize the computational results, only for the variant whose parameters were selected

by Irace as the best. Figures show that, in only 3 of the 27 instances in set A, the solution improves after 1200 seconds, with an average improvement of 0.29%. Similar results are observed for sets B and F, in which only 2 and 1 instances, respectively, achieve better solutions after 1200 seconds. The behavior of the method is analogous for sets E and P. In contrast, for sets M and Tai, all of which contain instances with more than 100 customers, improvements after 1200 seconds occur more frequently. Better solutions are found for 4 of the 5 instances in set M, for 9 of the 13 instances in set Tai. For these two sets, the average improvements are 0.93% and 1.49%, respectively.

Among the 106 instances considered, 96 contain fewer than 100 customers, whereas 10 contain 100 customers or more. Figures 6 and 7 show the evolution of the average incumbent solution value as a function of time for these two groups of instances. To prevent instances with larger objective values from exerting a disproportionate influence on the averages, the incumbent value of each instance was normalized by dividing it by the best incumbent obtained for that instance among the three parameter configurations after the full five-hour run. Figure 6, which displays only the first 30 minutes of the five-hour runs, indicates that the 1200-second time limit was sufficient for the instances with less than 100 customers. Improvements after the first 1200 seconds were observed in only a few instances, and the average incumbent curves quickly flatten, suggesting that near-convergence is reached within the allotted time. The figure also shows that GVNS exhibits virtually identical behavior under the three parameter configurations. In contrast, Figure 7 indicates that a 1200-second time limit is insufficient for the larger instances, as the average incumbent curves continue to decrease throughout the five-hour horizon. This behavior demonstrates that the search remains capable of finding improving solutions even after several hours of computation and suggests that additional gains could potentially be achieved through longer runs or stronger intensification strategies. Furthermore, even in this experiment, where each instance was solved only once, the ranking of the parameter configurations changes over time, indicating that a different time limit might have led Irace to select a different configuration as the best one. Nevertheless, the performance differences among the three GVNS variants remain small, highlighting the robustness of the proposed method with respect to parameter settings while also suggesting that large-scale instances remain a promising direction for further algorithmic development.

6 Conclusions and future work

In this work, we introduced a capacitated vehicle routing problem with polygonal neighborhoods (CVRPN) in which customers are represented by arbitrary polygonal regions, possibly non-convex. In contrast with classical VRP formulations, a route is defined not only by the sequence of customers to be visited but also by the selection of a visit point within each customer neighborhood. This additional geometric component substantially increases the complexity of the problem and requires the repeated solution of a MICQP during the search process.

To address this challenge, we proposed a block coordinate descent (BCD) procedure for computing visit points associated with a given visit sequence. Computational experiments showed that BCD provides solutions of the same quality as those obtained by solving the MIQCP formulations with a commercial solver while requiring only a tiny fraction of the computational

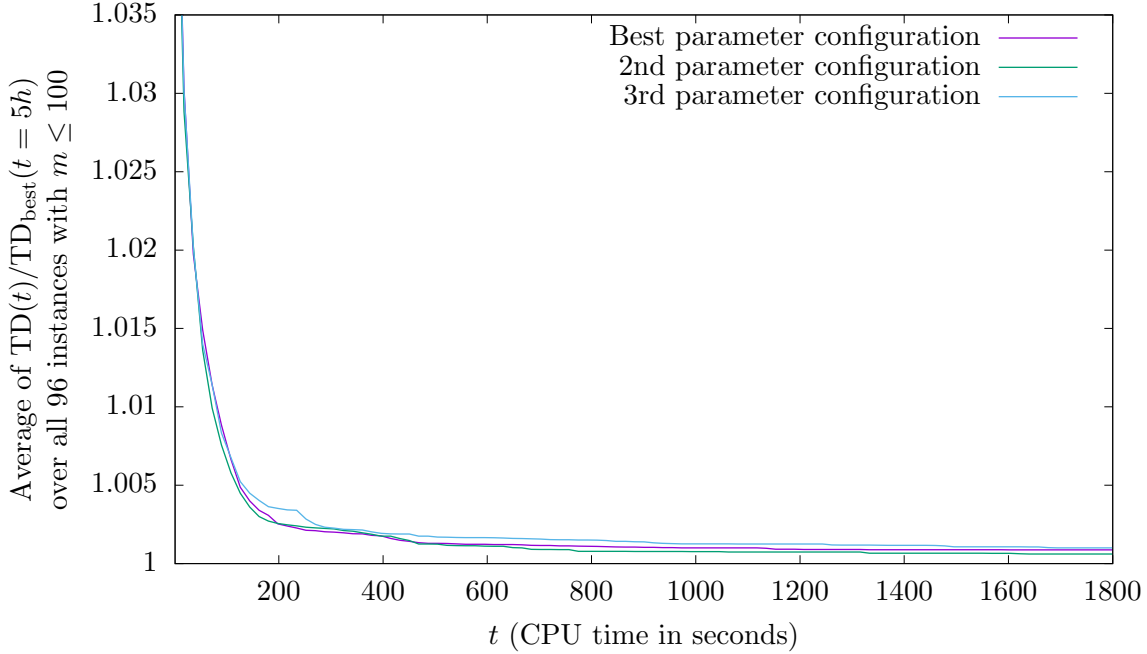


Figure 6: Evolution of the average incumbent solution over time for the three best parameter configurations identified by Irace. To give equal weight to all instances in the computation of the average, the incumbent value of each instance is normalized by the best solution found for that instance among the three configurations. The figure reports results for instances containing up to 100 customers.

effort, often reducing the solution time by several orders of magnitude. This result makes the repeated evaluation of candidate routes feasible within a metaheuristic framework. Therefore, a central contribution of the proposed framework is the BCD route-evaluation procedure, which enables the efficient optimization of visit points during the search process and makes large-scale GVNS exploration computationally tractable.

Building on this route-evaluation mechanism, we developed a GVNS metaheuristic tailored to the CVRPN. The method combines several intra-route and inter-route neighborhood structures and was calibrated automatically using Irace. Extensive experiments on both randomly generated instances and large-scale instances derived from CVRPLib benchmarks demonstrated the effectiveness of the proposed approach. For the small- and medium-scale instances, the method matched all solutions proven optimal by CPLEX and frequently improved the best solutions obtained by the exact solver within the prescribed time limit. For the large-scale benchmark set, the proposed method improved the CVRP-based upper bound in all instances, yielding an average improvement of 5.34%.

The analysis of the incumbent trajectories reveals a marked difference between small- and large-scale instances. For instances with up to 100 customers, the average incumbent value stabilizes within the 1200-second (20-minute) horizon considered in the experiments. For larger

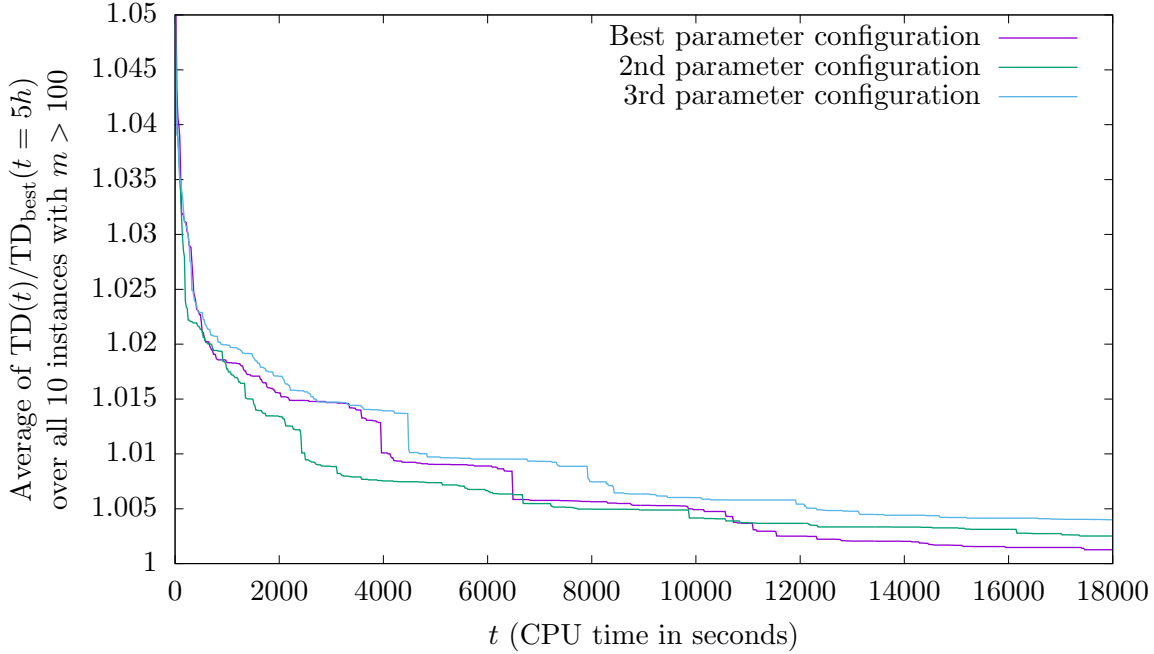


Figure 7: Evolution of the average incumbent solution over time for the three best parameter configurations identified by Irace. To give equal weight to all instances in the computation of the average, the incumbent value of each instance is normalized by the best solution found for that instance among the three configurations. The figure reports results for instances containing more than 100 customers.

instances, however, the average incumbent value continues to decrease throughout the entire 18000-second (5-hour) horizon. These results suggest that the proposed GVNS is able to approach convergence on the smaller instances, whereas additional computational effort or stronger intensification strategies may still yield significant improvements on the largest instances.

Overall, the results indicate that the combination of GVNS and BCD constitutes an effective framework for solving routing problems with geometric neighborhoods. Moreover, the proposed BCD procedure is independent of the surrounding metaheuristic and may therefore be incorporated into alternative search strategies. The mathematical model, benchmark instances, and software developed in this work provide a foundation for future research on routing problems with neighborhoods.

Several directions deserve further investigation. Future work includes the development of exact and matheuristic approaches, the study of richer variants involving time windows, heterogeneous fleets, or multiple depots, and the extension of the framework to three-dimensional neighborhoods motivated by aerial and robotic applications. Another promising direction is the design of specialized neighborhood structures that jointly modify customer sequences and visit-point locations. Finally, the incumbent trajectories observed for large-scale instances suggest that investigating longer-run search strategies, adaptive stopping criteria, and more powerful

intensification mechanisms may lead to further improvements in solution quality.

Acknowledgment: We sincerely thank Marcelo Marazzi for his careful editing and helpful suggestions that enhanced the quality of the written text in English.

References

- [1] E. Aarts and J. K. Lenstra (eds.). *Local Search in Combinatorial Optimization*. Princeton University Press, Princeton, NJ, 2003.
- [2] A. Alnaggar, F. Gzara, and J. H. Bookbinder. Crowdsourced delivery: A review of platforms and academic literature. *Omega*, 98:102139, 2021.
- [3] V. S. Amaral, R. Andreani, E. G. Birgin, D. S. Marcondes, and J. M. Martínez. On complexity and convergence of high-order coordinate descent algorithms for smooth nonconvex box-constrained minimization. *Journal of Global Optimization*, 84(3):527–561, 2022.
- [4] E. M. Arkin and R. Hassin. Approximation algorithms for the geometric covering salesman problem. *Discrete Applied Mathematics*, 55(3):197–218, 1994.
- [5] B. Behdani and J. C. Smith. An integer-programming-based approach to the close-enough traveling salesman problem. *INFORMS Journal on Computing*, 26(3):415–432, 2014.
- [6] S. N. Bezerra, S. R. de Souza, and M. J. F. Souza. A GVNS algorithm for solving the multi-depot vehicle routing problem. *Electronic Notes in Discrete Mathematics*, 66:167–174, 2018.
- [7] E. G. Birgin and J. M. Martínez. Block coordinate descent for smooth nonconvex constrained minimization. *Computational Optimization and Applications*, 83(1):1–27, 2022.
- [8] E. G. Birgin and R. S. Matos. Solving the capacitated vehicle routing problem with polygonal neighborhoods using a metaheuristic enhanced with block coordinate descent – Supplementary material. Technical Report MCDO010726, University of São Paulo, São Paulo, SP, Brazil, 2026.
- [9] F. Carrabs, C. Cerrone, R. Cerulli, and C. D’Ambrosio. Improved upper and lower bounds for the close enough traveling salesman problem. In *Green, Pervasive, and Cloud Computing*, pages 165–177. Springer International Publishing, 2017.
- [10] F. Carrabs, C. Cerrone, R. Cerulli, and M. Gaudioso. A novel discretization scheme for the close enough traveling salesman problem. *Computers & Operations Research*, 78:163–171, 2017.
- [11] G. Clarke and J. W. Wright. Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4):568–581, 1964.

- [12] W. P. Coutinho, R. Q. do Nascimento, A. A. Pessoa, and A. Subramanian. A branch-and-bound algorithm for the close-enough traveling salesman problem. *INFORMS Journal on Computing*, 28(4):752–765, 2016.
- [13] G. B. Dantzig and J. H. Ramser. The truck dispatching problem. *Management Science*, 6(1):80–91, 1959.
- [14] J. de Armas, B. Melián-Batista, J. A. Moreno-Pérez, and J. Brito. GVNS for a real-world rich vehicle routing problem with time windows. *Engineering Applications of Artificial Intelligence*, 42:45–56, 2015.
- [15] A. Di Placido, C. Archetti, and C. Cerrone. A genetic algorithm for the close-enough traveling salesman problem with application to solar panels diagnostic reconnaissance. *Computers & Operations Research*, 145:105831, 2022.
- [16] A. Di Placido, C. Archetti, C. Cerrone, and B. Golden. The generalized close enough traveling salesman problem. *European Journal of Operational Research*, 310(3):974–991, 2023.
- [17] J. Dong, N. Yang, and M. Chen. Heuristic approaches for a TSP variant: The automatic meter reading shortest tour problem. In E. K. Baker, A. Joseph, A. Mehrotra, and M. A. Trick, editors, *Extending the Horizons: Advances in Computing, Optimization, and Decision Technologies*, volume 37 of *Operations Research / Computer Science Interfaces Series*, pages 145–163. Springer US, Boston, MA, 2007.
- [18] M. M. Flood. The traveling-salesman problem. *Operations Research*, 4(1):61–75, 1956.
- [19] M. Gendreau, G. Laporte, and F. Semet. The covering tour problem. *Operations Research*, 45(4):568–576, 1997.
- [20] B. Golden, X. Wang, and E. Wasil. *The Evolution of the Vehicle Routing Problem—A Survey of VRP Research and Practice from 2005 to 2022*, pages 1–64. Springer Nature Switzerland, 2023.
- [21] D. J. Gulczynski, J. W. Heath, and C. C. Price. The close enough traveling salesman problem: A discussion of several heuristics. In F. B. Alt, M. C. Fu, and B. L. Golden, editors, *Perspectives in Operations Research: Papers in Honor of Saul Gass’ 80th Birthday*, volume 36 of *Operations Research / Computer Science Interfaces Series*, pages 271–283. Springer US, Boston, MA, 2006.
- [22] P. Hansen and N. Mladenović. Variable neighborhood search: Principles and applications. *European Journal of Operational Research*, 130(3):449–467, 2001.
- [23] P. Hansen, N. Mladenović, and J. A. Moreno Pérez. Variable neighbourhood search: methods and applications. *Annals of Operations Research*, 175(1):367–407, 2009.
- [24] C. Haslett. *Essentials of Radio Wave Propagation*. Cambridge University Press, Cambridge, UK, 2007.

- [25] S. Hert. 2D polygon partitioning. In *CGAL User and Reference Manual*. CGAL Editorial Board, 6.1 edition, 2025.
- [26] J. Kytöjoki, T. Nuortio, O. Bräysy, and M. Gendreau. An efficient variable neighborhood search heuristic for very large scale vehicle routing problems. *Computers & Operations Research*, 34(9):2743–2757, 2007.
- [27] G. Laporte. Fifty years of vehicle routing. *Transportation Science*, 43(4):408–416, 2009.
- [28] G. Laporte, M. Desrochers, and Y. Nobert. Two exact algorithms for the distance-constrained vehicle routing problem. *Networks*, 14(1):161–172, 1984.
- [29] T. V. Le, A. Stathopoulos, T. Van Woensel, and S. V. Ukkusuri. Supply, demand, operations, and management of crowd-shipping services: A review and empirical evidence. *Transportation Research Part C: Emerging Technologies*, 103:83–103, 2019.
- [30] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, and T. Stützle. The Irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.
- [31] M. Masmoudi, R. Borchani, and B. Jarboui. Generalized variable neighborhood search algorithm for vehicle routing problem with time windows and synchronization. *Computers & Operations Research*, 183:107193, 2025.
- [32] W. K. Mennell. *Heuristics for Solving Three Routing Problems: Close-Enough Traveling Salesman Problem, Close-Enough Vehicle Routing Problem, Sequence-Dependent Team Orienteering Problem*. PhD thesis, University of Maryland, College Park, MD, USA, 2009. Doctoral dissertation.
- [33] N. Mladenović and P. Hansen. Variable neighborhood search. *Computers & Operations Research*, 24(11):1097–1100, 1997.
- [34] N. Mladenović, R. Todosijević, and D. Urošević. An efficient GVNS for solving traveling salesman problem with time windows. *Electronic Notes in Discrete Mathematics*, 39:83–90, 2012.
- [35] L. D. P. Procópio. *Roteamento de drones autônomo: um estudo algorítmico*. PhD thesis, Centro de Ciências Exatas e da Terra, Universidade Federal do Rio Grande do Norte, Avenida Senador Salgado Filho, s/n, Lagoa Nova, Natal, RN, Brazil, 2020. Doctoral dissertation.
- [36] S. S. Shapiro and M. B. Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3/4):591–611, 1965.
- [37] R. Shuttlesworth, B. L. Golden, S. Smith, and E. Wasil. Advances in meter reading: Heuristic solution of the close enough traveling salesman problem over a street network. In B. Golden, S. Raghavan, and E. Wasil, editors, *The Vehicle Routing Problem: Latest Advances and New Challenges*, volume 43 of *Operations Research / Computer Science Interfaces Series*, pages 487–501. Springer US, Boston, MA, 2008.

- [38] M. Tadaros, A. Sifaleras, and A. Migdalas. A variable neighborhood search approach for solving a real-world hierarchical multi-echelon vehicle routing problem involving HCT vehicles. *Computers & Operations Research*, 165:106594, 2024.
- [39] R. Todosijević, S. Hanafi, D. Urošević, B. Jarboui, and B. Gendron. A general variable neighborhood search for the swap-body vehicle routing problem. *Computers & Operations Research*, 78:468–479, 2017.
- [40] P. Toth and D. Vigo, editors. *The Vehicle Routing Problem*. SIAM Monographs on Discrete Mathematics and Applications. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2002.
- [41] P. Toth and D. Vigo, editors. *Vehicle Routing: Problems, Methods, and Applications*. MOS-SIAM Series on Optimization. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2nd edition, 2014.
- [42] X. Wang, B. Golden, and E. Wasil. A Steiner zone variable neighborhood search heuristic for the close-enough traveling salesman problem. *Computers & Operations Research*, 101:200–219, 2019.
- [43] C. D. J. Waters. Expanding the scope of linear programming solutions for vehicle scheduling problems. *Omega*, 16(6):577–583, 1988.
- [44] F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.
- [45] S. J. Wright. Coordinate descent algorithms. *Mathematical Programming*, 151(1):3–34, 2015.
- [46] Z. Yang, M.-Q. Xiao, Y.-W. Ge, D.-L. Feng, L. Zhang, H.-F. Song, and X.-L. Tang. A double-loop hybrid algorithm for the traveling salesman problem with arbitrary neighbourhoods. *European Journal of Operational Research*, 265(1):65–80, 2018.
- [47] B. Yuan, M. Orlowska, and S. Sadiq. On the optimal robot routing problem in wireless sensor networks. *IEEE Transactions on Knowledge and Data Engineering*, 19(9):1252–1261, 2007.
- [48] W. Zhang, J. J. Sauppe, and S. H. Jacobson. Results for the close-enough traveling salesman problem with a branch-and-bound algorithm. *Computational Optimization and Applications*, 85(2):369–407, 2023.